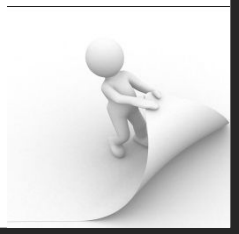
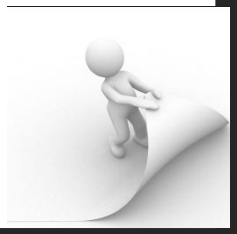
C++11新特性之五：新特性补充与C++11代码移植

张锦成 2015/10/16



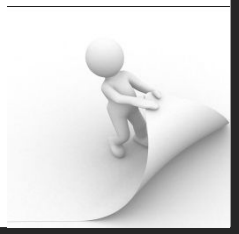
Outline

- final/override控制
- 类的非静态成员的sizeof
- C++11和C++98/03不兼容的地方
- C++11新特性回顾



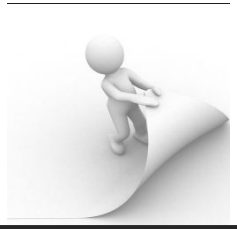
1

final/override控制



final/override

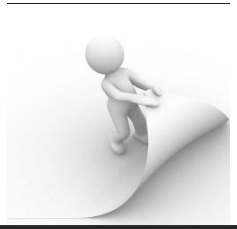
- 类继承中，如何阻止虚成员函数继续往下重写
- 派生类中如何显示指明并且要求重写基类的虚函数



final

类继承中，如何阻止虚成员函数继续往下重写

```
class MathObject{
    public:
        virtual double Arith() = 0;
        virtual void print() = 0;
};
class Printable: public MathObject{
    public:
        double Arith() = 0;
        void print() {
            cout << "Output is : " << Arith() << endl;
        }
};

class Add2: public Printable{
    public:
        Add2(double a, double b): x(a), y(b){}
        double Arith() {return x+y;}
    private:
        double x, y;
};
class Mul3: public Printable{
    public:
        Mul3(double a, double b, double c): x(a), y(b), z(c){}
        double Arith() {return x*y*z;}
        //void print(){cout << "Mul3: " << Arith() << endl;}
    private:
        double x, y, z;
};
```



final

- final修饰符可以阻止虚函数在类继承体系中往下重写

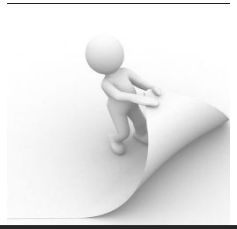
```
class Printable: public MathObject{
    public:
        double Arith() = 0;
        void print() final {
            cout << "Output is : " << Arith() << endl;
        }
};
```

override

- 派生类中如何显示指明并且要求重写基类的虚函数

有时需要直观知道类成员函数到底是不是虚函数，而不是一直往上去查找父类

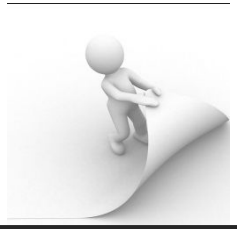
有时可能重载了基类的虚函数，而不是重写了虚函数



override

override修饰符可以强制派生类要实现基类的同名虚函数

```
struct Base{
    virtual void Turing() = 0;
    virtual void Dijkstra() = 0;
    virtual void VNeumann(int g) = 0;
    virtual void DKnuth() const;
    void print();
};

struct DerivedMid: public Base{
    // void VNeumann(double g);
    // 接口被注释了，曾想多一个版本的VNeumann函数
};

struct DerivedTop: public DerivedMid {
    void Turing() override;
    void Dikjstra() override;  // error
    void VNeumann(double g) override; // error
    void DKnuth() override; // error
    void print() override; // error
};
```

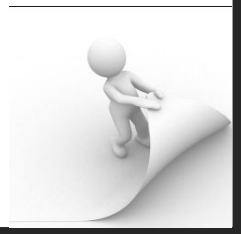


final/override

- final可用于基类或派生类，不过用在基类里就失去了虚函数的意义
- override只能用在派生类，因为是要重写基类的同名虚函数

2

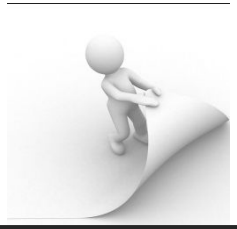
类的非静态成员的sizeof



sizeof

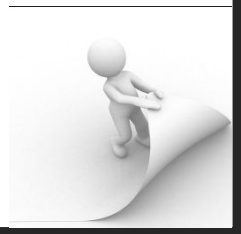
非静态成员取sizeof，提升代码可读性与编写方便性

```
class Test{
    public:
        int hand;
        static double ear;
};

double Test::ear = 1.0;

int main()
{
    Test test;
    cout << sizeof(test.hand) << endl;
    cout << sizeof(Test::ear) << endl;
    // C++98标准不支持, C++11支持
    cout << sizeof(Test::hand) << endl;
    // C++98可以使用如下方法
    cout << sizeof(((Test*)0)->hand) << endl;
    return 0;
}
```



3

C++11和C++98/03不兼容的地方



新关键字

- 如果C++98/03代码用这些新关键字作为变量名，会被C++11编译器视为非法代码

alignas, alignof, char16_t, char32_t, constexpr,

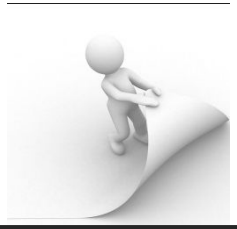
decltype, noexcept, nullptr, static_assert, thread_local

{}初始化列表防止类型收窄

✓ 类型收窄：类型转换过程中会出现精度损失

```
int i {1.0};
int g[] = {1.0, 2};
```

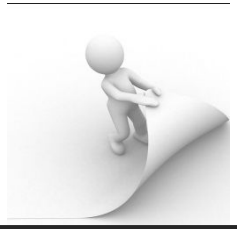
```
2-8-1.cpp:68:12: warning: narrowing conversion of '1.0e+0' from 'double' to 'int
' inside { } [-Wnarrowing]
  int i {1.0};
            ^
2-8-1.cpp:69:19: warning: narrowing conversion of '1.0e+0' from 'double' to 'int
' inside { } [-Wnarrowing]
  int g[] = {1.0, 2};
                  ^
```



>> 双右尖括号

- 嵌套模板中出现右移

```
X< Y<1>>2> > x; // 只能在C++98中编译通过

X<Y<(1>>2)>> x; // 只能在C++11中编译通过
```



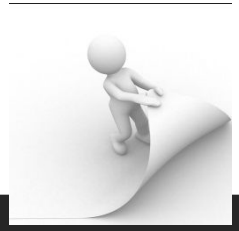
auto

- C++11中auto不再作为存储类型的修饰符，而用于自动类型推导

```
auto int i = 1;
```

References

1. 《深入理解C++11: C++11新特性解析与应用》第3章和附录A
2. C++11新特性: http://www.ituring.com.cn/article/39533



THANK
YOU!